# Алгоритмы и структуры данных — 1 семестр · ITMO IS

Шпаргалка: Анализ сложности · Сортировки · Структуры данных · Бинарный поиск · ДП

## Содержание


1. Блок 1: Анализ сложности ..... 2
   1.1. Асимптотические нотации ..... 2
   1.2. Мастер-теорема ..... 2
2. Блок 2: Сортировки ..... 3
   2.1. Сортировка вставками ..... 3
      2.1.1. Анализ сложности ..... 3
      2.1.2. Псевдокод ..... 3
      2.1.3. Трассировка ..... 3
   2.2. Сортировка слиянием ..... 3
      2.2.1. Анализ сложности ..... 3
      2.2.2. Псевдокод ..... 3
      2.2.3. Трассировка ..... 3
   2.3. Быстрая сортировка ..... 3
      2.3.1. Анализ сложности ..... 3
      2.3.2. Псевдокод ..... 3
      2.3.3. Трассировка ..... 4
   2.4. Сортировка подсчётом ..... 4
      2.4.1. Анализ сложности ..... 4
      2.4.2. Псевдокод ..... 4
      2.4.3. Трассировка ..... 4
   2.5. Цифровая сортировка (Radix LSD) ..... 4
      2.5.1. Анализ сложности ..... 4
      2.5.2. Псевдокод ..... 4
      2.5.3. Трассировка ..... 4
3. Блок 3: Структуры данных ..... 4
   3.1. Стек (LIFO) ..... 4
   3.2. Очередь (FIFO) ..... 4
   3.3. Связные списки ..... 4
   3.4. Дек и очередь с минимумом ..... 4
      3.4.1. Псевдокод ..... 4
      3.4.2. Анализ сложности ..... 4
      3.4.3. Трассировка ..... 4
4. Блок 4: Пирамида (Heap) ..... 4
   4.1. Свойства и индексы ..... 5
   4.2. Базовые операции ..... 5
      4.2.1. Псевдокод ..... 5
   4.3. Построение за O(n) ..... 5
   4.4. Пирамидальная сортировка ..... 5
   4.5. Очередь с приоритетом ..... 5
5. Блок 5: Бинарный поиск ..... 5
   5.1. lower_bound (первый >= target) ..... 5
   5.2. Поиск по ответу ..... 5
   5.3. Вещественный бинарный поиск ..... 5
6. Блок 6: Динамическое программирование ..... 5
   6.1. Кузнечик ..... 6
   6.2. Рюкзак 0/1 ..... 6
   6.3. НВП (Longest Increasing Subsequence) ..... 6
   6.4. Редакционное расстояние (Левенштейн) ..... 6
   6.5. ДП по подмножествам (bitmask DP / TSP) ..... 6


# 1. Блок 1: Анализ сложности

## 1.1. Асимптотические нотации

- $O(f(n))$: $\exists c > 0, n_0$ такие, что $T(n) \le c \cdot f(n)$ при всех $n > n_0$. **Верхняя оценка.**
- $\Omega(f(n))$: $T(n) \ge c \cdot f(n)$. **Нижняя оценка.**
- $\Theta(f(n))$: одновременно $O$ и $\Omega$. **Точная оценка.**

**Правила:** $O(f+g) = O(\max(f,g))$; $O(c \cdot f) = O(f)$.

**Иерархия:** $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

## 1.2. Мастер-теорема

$T(n) = a \cdot T(n/b) + f(n)$, $a \ge 1$, $b > 1$. Сравниваем $f(n)$ с $n^{\log_b a}$:

- **Случай 1:** $f(n) = O\big(n^{\log_b a - \varepsilon}\big) \Rightarrow T(n) = \Theta\big(n^{\log_b a}\big)$ [тяжёлые листья]
- **Случай 2:** $f(n) = \Theta\big(n^{\log_b a}\big) \Rightarrow T(n) = \Theta\big(n^{\log_b a} \cdot \log n\big)$ [равновесие]
- **Случай 3:** $f(n) = \Omega\big(n^{\log_b a + \varepsilon}\big)$ и $af(n/b) \le cf(n) \Rightarrow T(n) = \Theta(f(n))$ [тяжёлый корень]

**Примеры:**

- Merge Sort: $T(n) = 2T(n/2) + O(n)$, $a = 2, b = 2$, $\log_2 2 = 1 \to$ случай 2 $\to \Theta(n \log n)$
- Binary Search: $T(n) = T(n/2) + O(1)$, $\log_2 1 = 0 \to$ случай 2 $\to \Theta(\log n)$
- $T(n) = 4T(n/2) + O(n)$: $\log_2 4 = 2 > 1 \to$ случай 1 $\to \Theta(n^2)$

# 2. Блок 2: Сортировки

## 2.1. Сортировка вставками

### *2.1.1. Анализ сложности*

**Время:** $O(n^2)$ worst/avg (обратный порядок $\Rightarrow$ сумма $1 + 2 + ... + (n-1) = n(n-1)/2$ сдвигов), $O(n)$ best (уже отсортирован). **Память:** $O(1)$ in-place. **Устойчива.**

### *2.1.2. Псевдокод*

```
insertionSort(A, n):
  for i = 1 to n-1:
    key = A[i]; j = i-1
    while j >= 0 and A[j] > key:
      A[j+1] = A[j]; j--
    A[j+1] = key
```

### *2.1.3. Трассировка*

`[5, 2, 4, 1, 3]`:

- `i=1, key=2`: сдвиг 5 → `[2, 5, 4, 1, 3]`
- `i=2, key=4`: сдвиг 5 → `[2, 4, 5, 1, 3]`
- `i=3, key=1`: сдвиг 5,4,2 → `[1, 2, 4, 5, 3]`
- `i=4, key=3`: сдвиг 5,4 → `[1, 2, 3, 4, 5]`

## 2.2. Сортировка слиянием

### *2.2.1. Анализ сложности*

**Время:** $\Theta(n \log n)$. Дерево рекурсии глубиной $\log_2 n$; на каждом из $\log n$ уровней суммарно $n$ операций слияния $\to n \cdot \log n$. **Память:** $O(n)$ (буфер для слияния). **Устойчива.**

### *2.2.2. Псевдокод*

```
mergeSort(A, lo, hi):
  if lo >= hi: return
  mid = (lo+hi)/2
  mergeSort(A, lo, mid)
  mergeSort(A, mid+1, hi)
  merge(A, lo, mid, hi)   // O(n) за проход

merge(A, lo, mid, hi):
  tmp = copy A[lo..hi]
  li=lo, ri=mid+1, k=lo
  while li<=mid and ri<=hi:
    if tmp[li]<=tmp[ri]: A[k++]=tmp[li++]
    else:                A[k++]=tmp[ri++]
  copy remaining
```

### *2.2.3. Трассировка*

`[5, 1, 4, 2, 3]`:

- Разбиение: `[5,1]` | `[4,2,3]`
- Sort L: `[1,5]`
- Sort R: `[4]` merge `[2,3]` = `[2,3,4]`
- Merge `[1,5]` + `[2,3,4]` → `[1,2,3,4,5]`

## 2.3. Быстрая сортировка

### *2.3.1. Анализ сложности*

**Время:** $\Theta(n \log n)$ среднее, $O(n^2)$ худшее (уже отсортирован + крайний pivot). Среднее: $T(n) = 2T(n/2) + n \to$ случай 2 мастер-теоремы. **Память:** $O(\log n)$ стек ($O(n)$ худш.). **Нестабильна.**

Quickselect: только одна ветвь рекурсии $\to T(n) = n + n/2 + n/4 + ... = 2n \to O(n)$ avg.

### *2.3.2. Псевдокод*

```
quickSort(A, lo, hi):
  if lo >= hi: return
  p = partition(A, lo, hi)
  quickSort(A, lo, p-1)
  quickSort(A, p+1, hi)

partition(A, lo, hi):   // Lomuto
```

```
  pivot=A[hi]; i=lo-1
  for j=lo to hi-1:
    if A[j]<=pivot: swap A[++i], A[j]
  swap A[i+1], A[hi]; return i+1

quickselect(A, lo, hi, k):
  if lo==hi: return A[lo]
  p = partition(A, lo, hi)
  if k==p:   return A[p]
  elif k<p:  return quickselect(A, lo, p-1,
k)
  else:      return quickselect(A, p+1, hi,
k)
```

#### *2.3.3. Трассировка*

`[3,7,8,2,4]`, pivot=`A[4]`=4:

- `j=0`: 3≤4, swap(`A[0]`,`A[0]`), `i=0` → `[3,7,8,2,4]`
- `j=1`: 7>4, skip
- `j=2`: 8>4, skip
- `j=3`: 2≤4, swap(`A[1]`,`A[3]`), `i=1` → `[3,2,8,7,4]`
- End: swap(`A[2]`,`A[4]`) → `[3,2,4,7,8]`. Pivot 4 → pos 2.

### 2.4. Сортировка подсчётом

#### *2.4.1. Анализ сложности*

**Время:** $\Theta(n+k)$. Три прохода: подсчёт $O(n)$, префиксные суммы $O(k)$, расстановка $O(n)$. **Память:** $O(k)$ (массив count). **Устойчива.** Только целые $\geq 0$.

#### *2.4.2. Псевдокод*

```
countingSort(A, n, k):
  count[0..k] = {0}
  // Фаза 1: подсчёт
  for i=0..n-1: count[A[i]]++
  // Фаза 2: префиксные суммы
  for i=1..k: count[i] += count[i-1]
  // Фаза 3: расстановка (справа налево =
устойчива)
  for i=n-1 downto 0:
    out[--count[A[i]]] = A[i]
```

#### *2.4.3. Трассировка*

`[4,1,4,1,3]`, k=5:

- Freq: 1→2, 3→1, 4→2
- Prefix: `[0,2,2,3,5]`
- Fill←: 3→out[2], 1→out[1], 4→out[4], 1→out[0], 4→out[3]
- Результат: `[1,1,3,4,4]`

### 2.5. Цифровая сортировка (Radix LSD)

#### *2.5.1. Анализ сложности*

**Время:** $\Theta(d \cdot (n+k))$, $d$ — число разрядов, $k=10$ для десятичных. **Память:** $O(n+k)$. **Устойчива.** Каждый проход — устойчивая сортировка подсчётом.

#### *2.5.2. Псевдокод*

```
radixSort(A, n, d):
  for pass = 0 to d-1:
    стабильно отсортировать A по разряду pass
    // digit = (A[i] / base^pass) % base
```

#### *2.5.3. Трассировка*

`[170, 45, 75, 90, 802, 24, 2, 66]`:

- По ед.: `[170,90,802,2,24,45,75,66]`
- По дес.: `[802,2,24,45,66,170,75,90]`
- По сот.: `[2,24,45,66,75,90,170,802]`

## 3. Блок 3: Структуры данных

### 3.1. Стек (LIFO)

Вставка и извлечение с одного конца. Реализация на массиве (top-указатель) или связном списке.

```
push(x):    stack[++top] = x     // O(1)
pop():      return stack[top--]  // O(1)
peek():     return stack[top]    // O(1)
is_empty(): return top == -1
```

**Память:** $O(n)$. **Применение:** DFS, скобочные выражения, undo/redo.

### 3.2. Очередь (FIFO)

Кольцевой буфер: `front`, `back` по модулю N.

```
enqueue(x): q[back] = x; back=(back+1)%N
dequeue():  x=q[front]; front=(front+1)%N;
return x
```

**Применение:** BFS, планирование задач, Producer-Consumer.

### 3.3. Связные списки

**Односвязный:** узел = `{val, next}`. insert_head $O(1)$, остальное $O(n)$.

**Двусвязный:** узел = `{val, next, prev}`. Вставка/удаление при известном узле — $O(1)$.

```
insert_after(node, val):
  new = Node(val)
  new.next=node.next; new.prev=node
  if node.next: node.next.prev=new
  node.next=new

delete(node):
  node.prev.next=node.next
  node.next.prev=node.prev
```

### 3.4. Дек и очередь с минимумом

**Дек:** push/pop с обоих концов за $O(1)$.

Очередь с минимумом — монотонная дека (кандидаты на min в возрастающем порядке):

#### *3.4.1. Псевдокод*

```
enqueue(x):                // Монотонная дека
  while !dmin.empty() and dmin.back()>x:
    dmin.pop_back()
  dmin.push_back(x); q.push_back(x)

dequeue():
  val = q.pop_front()
  if dmin.front()==val: dmin.pop_front()
  return val

get_min(): return dmin.front()  // O(1)
```

#### *3.4.2. Анализ сложности*

Все операции $O(1)$ амортизированно. Каждый элемент помещается/удаляется из деки $\leq 1$ раза.

#### *3.4.3. Трассировка*

enqueue 5, 3, 1, 4:

- dmin: `[5]`→`[3]`→`[1]`→`[1,4]` (3 вытесняет 5; 1 вытесняет 3)
- `get_min()` = 1 (всегда front)

# 4. Блок 4: Пирамида (Heap)

## 4.1. Свойства и индексы

Max-heap: `heap[parent] >= heap[child]`.

Индексы (с нуля): `parent(i)=(i-1)/2`, `left=2i+1`, `right=2i+2`.

## 4.2. Базовые операции

### *4.2.1. Псевдокод*

```
sift_up(i):
  while i>0 and heap[parent(i)] < heap[i]:
    swap heap[i], heap[parent(i)]
    i = parent(i)                  // O(log
n)

sift_down(i, n):
  while 2i+1 < n:
    largest = i
    if heap[2i+1]>heap[largest]: largest=2i+1
    if 2i+2<n and heap[2i+2]>heap[largest]:
      largest=2i+2
    if largest==i: break
    swap heap[i], heap[largest]; i=largest

insert(x):    heap[n++]=x; sift_up(n-1)    //
O(log n)
extractMax(): max=heap[0]; heap[0]=heap[--n]
              sift_down(0,n); return
max     // O(log n)
peek_max():   return
heap[0]                // O(1)

deleteAt(i):  heap[i]=heap[--n]
              sift_up(i);
sift_down(i,n)    // O(log n)
```

## 4.3. Построение за O(n)

Запускаем `sift_down` от последнего нелистового узла (`i = n/2-1`) до корня.

```
buildHeap(A, n):
  for i = n/2-1 downto 0:
    sift_down(A, i, n)
```

**Доказательство $O(n)$:** листьев $\approx n/2$ (глубина 0), узлов на уровне $h$ — $n/2^{h+1}$, работа `sift_down` — $O(h)$. Сумма: $\sum_h h \cdot n/2^{h+1} = O(n)$ (сходящийся ряд).

## 4.4. Пирамидальная сортировка

```
heapSort(A, n):
  buildHeap(A, n)              // O(n)
  for i = n-1 downto 1:
    swap A[0], A[i]
    sift_down(A, 0, i)         // O(log n)
```

**Время:** $O(n \log n)$. **Память:** $O(1)$. **Нестабильна.** Не cache-friendly.

## 4.5. Очередь с приоритетом

На max-heap. **Применение:** Дейкстра, Прим, A*.

`insert(x, p)`: $O(\log n)$. `extractMax`: $O(\log n)$. `peek`: $O(1)$.

`increaseKey(i, k)`: `heap[i].p=k; sift_up(i)`.

`decreaseKey(i, k)`: `heap[i].p=k; sift_down(i,n)`.

# 5. Блок 5: Бинарный поиск

## 5.1. lower_bound (первый >= target)

```
lower_bound(A, n, target):
  lo=0, hi=n          // [lo, hi) —
полуоткрытый
  while lo < hi:
    mid = (lo+hi)/2
    if A[mid] < target: lo=mid+1
    else:               hi=mid
  return lo            // lo=n если не найден
```

**Инвариант:** `A[lo-1]<target<=A[hi]`. При выходе `lo=hi`=ответ.

`upper_bound` (первый $>$ x): заменить `<` на `<=`.

**Время:** $O(\log n)$. **Память:** $O(1)$.

## 5.2. Поиск по ответу

Монотонный предикат `check(x)`: False…False|True…True. Найти первый True:

```
lo=min_ans, hi=max_ans
while lo < hi:
  mid=(lo+hi)/2
  if check(mid): hi=mid
  else:          lo=mid+1
return lo
```

## 5.3. Вещественный бинарный поиск

```
lo=left_bound, hi=right_bound
repeat 100 times:   // или while hi-lo > eps
  mid=(lo+hi)/2
  if f(mid) >= 0: hi=mid
  else:           lo=mid
return (lo+hi)/2
```

**Пример:** $\sqrt{2}$ = bs на $[1,2]$, $f(x)=x^2-2$, 50 итераций дают точность $10^{-15}$.

# 6. Блок 6: Динамическое программирование

## 6.1. Кузнечик

1 или 2 шага, минимальная стоимость:

```
dp[0]=0
for i=1 to n:
  dp[i] = cost[i] + min(dp[i-1],
                  dp[i-2] if i>=2 else INF)
```

$O(n)$ время, $O(1)$ память с перекатыванием (хранить только 2 предыдущих).

## 6.2. Рюкзак 0/1

$n$ предметов, ёмкость $W$. `dp[w]` = макс. ценность при ёмкости $w$.

```
dp[0..W] = {0}
for i=1 to n:
  for w=W downto items[i].weight:
    dp[w] = max(dp[w],
                dp[w-weight[i]] + value[i])
```

**Время:** $O(nW)$. **Память:** $O(W)$. Проход справа налево обеспечивает 0/1 (не повторяем предмет).

## 6.3. НВП (Longest Increasing Subsequence)

`dp[i]` = длина НВП, заканчивающейся в `A[i]`.

```
// O(n²):
for i=0..n-1:
  dp[i]=1
  for j<i: if A[j]<A[i]: dp[i]=max(dp[i],
dp[j]+1)
ans = max(dp)

// O(n log n): терпение + бинарный поиск
tails = []  // tails[l] = наим. хвост НВП
длины l+1
for x in A:
  pos = lower_bound(tails, x)
  tails[pos] = x
ans = len(tails)
```

## 6.4. Редакционное расстояние (Левенштейн)

`dp[i][j]` = мин. операций (вставка, удаление, замена) для `s[0..i-1]`→`t[0..j-1]`.

```
dp[0][j]=j, dp[i][0]=i
for i=1..n, j=1..m:
  if s[i-1]==t[j-1]: dp[i][j]=dp[i-1][j-1]
  else: dp[i][j]=1+min(dp[i-1][j],
                       dp[i][j-1],
                       dp[i-1][j-1])
```

**Время:** $O(nm)$. **Память:** $O(nm)$ или $O(m)$ с перекатыванием строк.

Трассировка KITTEN→SITTING: `dp[6][7]=3` (2 замены + 1 вставка).

## 6.5. ДП по подмножествам (bitmask DP / TSP)

`dp[mask][v]` = мин. стоимость пути через все города в mask, финиш в $v$.

```
dp[1<<s][s] = 0   // start город s
for mask=1..(1<<n)-1:
  for v in mask:
    for u not in mask:
      nMask = mask | (1<<u)
      dp[nMask][u] = min(dp[nMask][u],
                         dp[mask][v] +
dist[v][u])
ans = min over v of dp[(1<<n)-1][v]
```

**Время:** $O(2^n \cdot n^2)$. **Память:** $O(2^n \cdot n)$. $n \leq 20$ на практике.